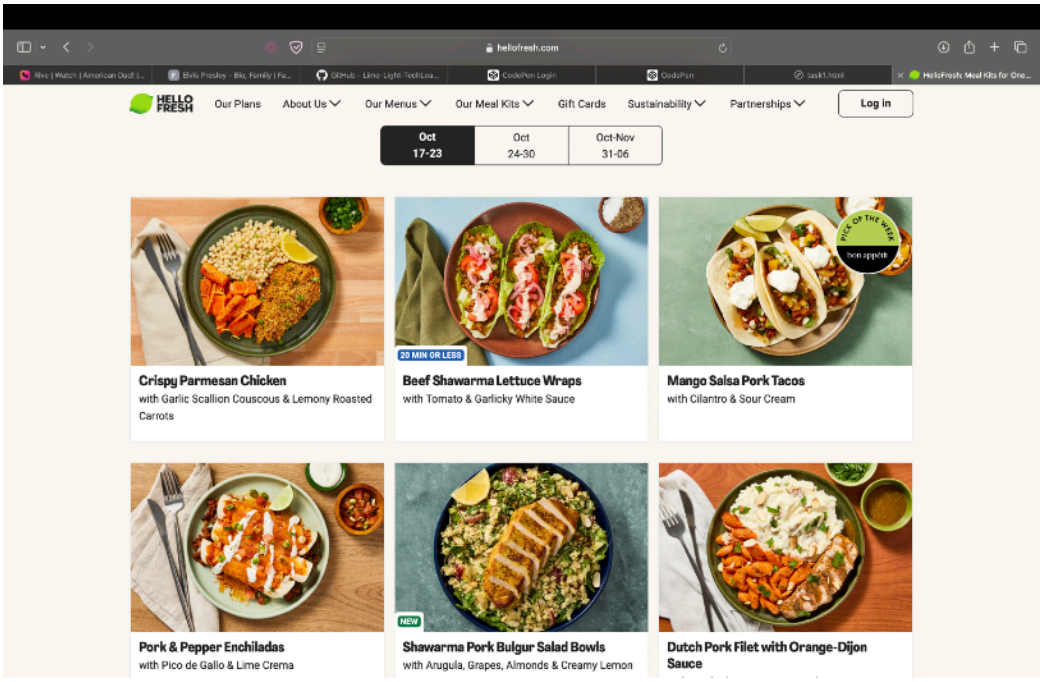
CSS 3D flip cards

For my project, I was inspired by meal kit services, in particular HelloFresh.

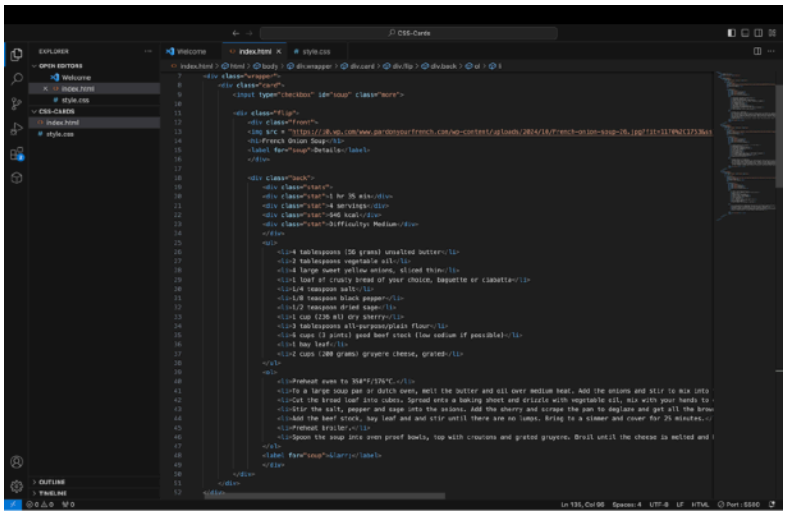


These meal kit services offer ingredients to be delivered at someones house on a weekly basis and recipes to make a meal of the ingredients provided. On the Hello Fresh website, when you click on the meal, nothing happens, they're just images. So my project takes the images displayed a step further, each meal is a card that flips over to reveal the full recipe without having to leave the page. I named my service "Box to Home".

HTML

When I studied the code provided I noted that each card is made of nested div containers. The outermost container is the card, which stays still and sets the size. Inside is the container that's responsible for flipping. This flipping container holds two child containers, the front and the back, which contain the actual content. Just before the flip container is a checkbox that acts as a switch that triggers the flip, this is how the website can make such movements without JavaScript.

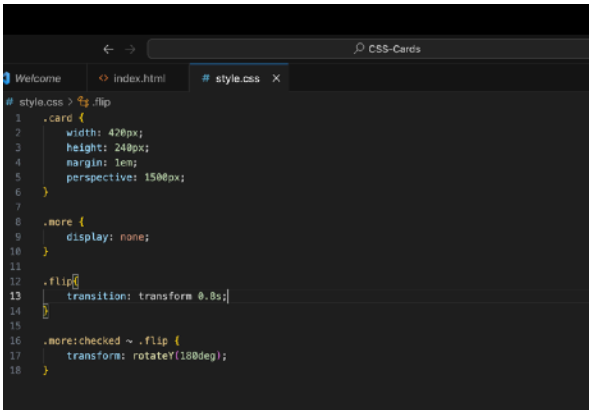
I began with a basic HTML page containing only the content, the title, image, stats and recipe. Then I gradually grouped it into nested containers.



CSS

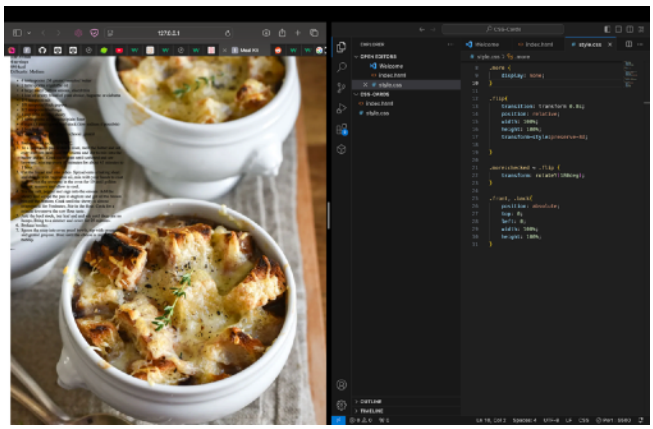
From reading the CSS code provided I determined that the flip needs 4 properties working together. First, perspective on the card sets how far the viewer is from the scene, without it the rotation would look flat.

Second, transform rotateY turns the flip container half a turn around its vertical axis. The transition property makes this happen gradually over 0.8 seconds instead of instantly.

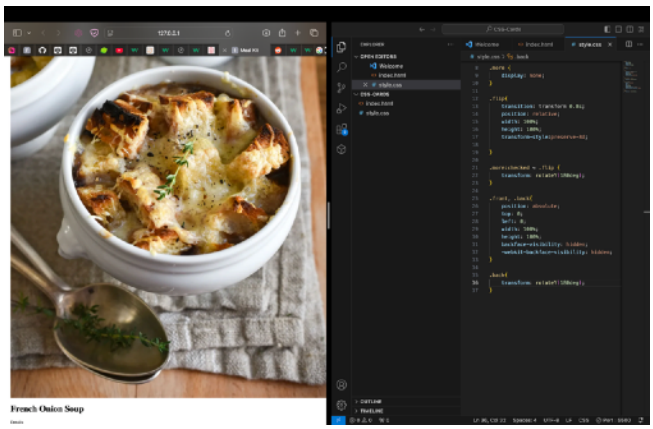


Third, transform-style preserve-3d tells the browser to keep the front and back as separate surfaces in 3D space, rather than flattening them into one image.

Lastly, backface-visibility being hidden make each face invisible once it turns away from the viewers.

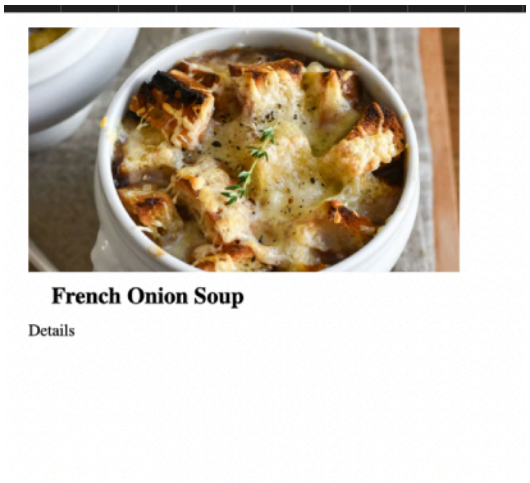


Before hiding backface visibility

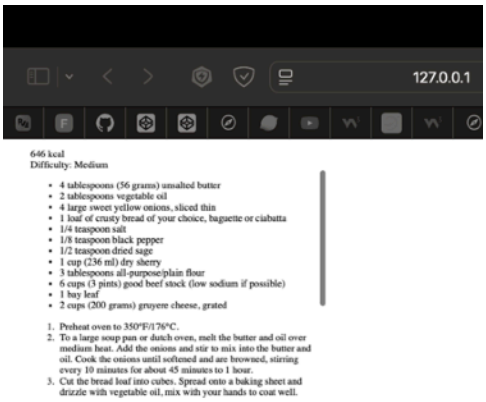


After hiding backface visibility

These four properties, I found to be essential to make the flipping mechanism work so I kept them exactly the same and the rest of the changes were personal style choices I made to make the website fit the theme I was going for. After using the .front img rule to size the photo to fit the top of the card, I was left with a good canvas to make some stylistic choices.



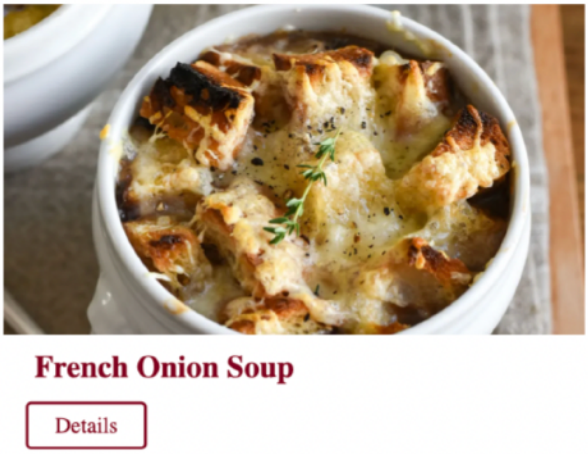
Front of the card. Flips when “Details” is clicked.



Back of the card. Arrow at the bottom flips back to the front.

With the help of color.adobe.com I landed on having burgundy #800020 and a warm cream colour #f5ecd7 to help evoke a feeling of cosines that comes with the idea of a home cooked meal, making the subscription service more appealing.

```css
.front label {
    display: inline-block;
    margin-left: 1em;
    padding: 0.4em 1.2em;
    border: 2px solid var(--burgundy);
    border-radius: 4px;
    color: var(--burgundy);
    cursor: pointer;
}
```



I made the decision to have the name of the meal sit below the picture instead of on top like the project provided. I wanted to have the image on full display so that it is more appealing to the user. However I was having an issue with the details button being at the bottom as well, I found it to be a bit clunky and visually distracting. So my idea was to have the details button appear upon hovering.



French Onion Soup

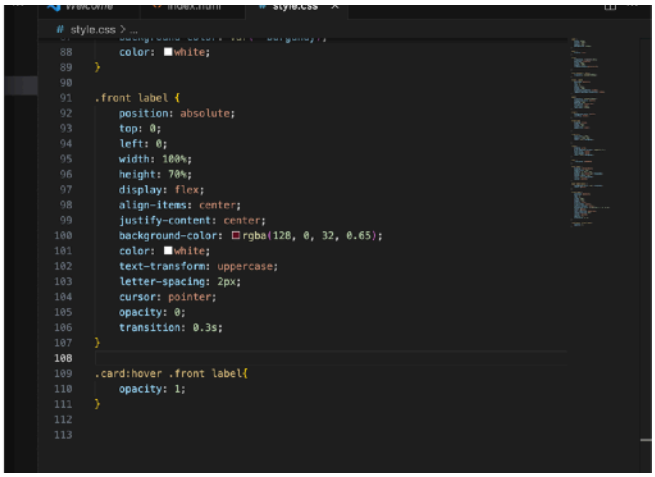


French Onion Soup

My first attempt placed the Details text directly on top of the photo but as the screenshots show, this was very hard to read against the busy image. To fix this I turned the button into a semi-transparent burgundy layer that covers the whole photo with “DETAILS” centred in white.



French Onion Soup



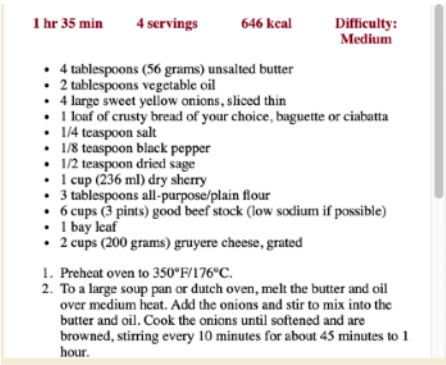
```
# style.css > ...
    color: white;
}

.front label {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 70%;
    display: flex;
    align-items: center;
    justify-content: center;
    background-color: rgba(128, 0, 32, 0.65);
    color: white;
    text-transform: uppercase;
    letter-spacing: 2px;
    cursor: pointer;
    opacity: 0;
    transition: 0.3s;
}

.card:hover .front label{
    opacity: 1;
}
```

It stays hidden until the user hovers over the card then fades in. This keeps the photo clear at first, makes the text easy to read and makes the whole image clickable.

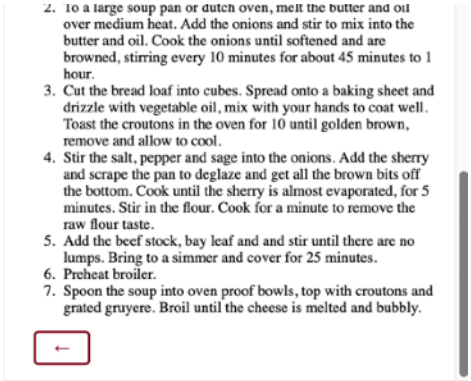
On the back, the cooking time, servings calories and difficulty sit in a row using css grid with four equal columns.



1 hr 35 min **4 servings** **646 kcal** **Difficulty: Medium**

- 4 tablespoons (56 grams) unsalted butter
- 2 tablespoons vegetable oil
- 4 large sweet yellow onions, sliced thin
- 1 loaf of crusty bread of your choice, baguette or ciabatta
- 1/4 teaspoon salt
- 1/8 teaspoon black pepper
- 1/2 teaspoon dried sage
- 1 cup (236 ml) dry sherry
- 3 tablespoons all-purpose/plain flour
- 6 cups (3 pints) good beef stock (low sodium if possible)
- 1 bay leaf
- 2 cups (200 grams) gruyere cheese, grated

1. Preheat oven to 350°F/176°C.
2. To a large soup pan or dutch oven, melt the butter and oil over medium heat. Add the onions and stir to mix into the butter and oil. Cook the onions until softened and are browned, stirring every 10 minutes for about 45 minutes to 1 hour.



2. To a large soup pan or dutch oven, melt the butter and oil over medium heat. Add the onions and stir to mix into the butter and oil. Cook the onions until softened and are browned, stirring every 10 minutes for about 45 minutes to 1 hour.
3. Cut the bread loaf into cubes. Spread onto a baking sheet and drizzle with vegetable oil, mix with your hands to coat well. Toast the croutons in the oven for 10 until golden brown, remove and allow to cool.
4. Stir the salt, pepper and sage into the onions. Add the sherry and scrape the pan to deglaze and get all the brown bits off the bottom. Cook until the sherry is almost evaporated, for 5 minutes. Stir in the flour. Cook for a minute to remove the raw flour taste.
5. Add the beef stock, bay leaf and and stir until there are no lumps. Bring to a simmer and cover for 25 minutes.
6. Preheat broiler.
7. Spoon the soup into oven proof bowls, top with croutons and grated gruyere. Broil until the cheese is melted and bubbly.

←

Building the other cards

Once the first card worked, I reused the same HTML structure for the other dishes. The CSS applies to all of them automatically because they share the same class names. The only change needed was giving each card's checkbox a unique id, with both labels pointing to it so that each card flips independently.

